

HW1

1.常量折叠转换 (Constant Propagation)

AST的子类都包含 `accept(ASTVisitor &v)` 函数，即调用对应的 `visitor` 访问该节点。

实现一个访问者 `ConstantPropagationVisitor`。重写其中的 `visit` 函数。由于 `accept` 是 `void` 类型，想要获得 `visit` 访问后返回的值，需要在访问者类中创建一个 `AST* newNode` 来存储返回值。

对于 `Program`，只需要处理其中的 `MainMethod` 后接到一个新的 `Program` 实例上并返回。

对于 `MainMethod`，枚举其中的语句，逐个处理后存入一个新的 `vector` 中，之后用新的 `vector` 新建一个 `MainMethod` 实例并返回。

对于语句 `Stm` 的两个子类 `Assign` 和 `Return`。`Assign` 分别先处理左右的 `Exp` 表达式，然后合成新的 `Assign` 并返回。`Return` 则把对应的 `Exp` 表达式处理后返回新的 `Return` 实例。

对于 `Exp` 的子类 `BinaryOp`, `UnaryOp`, `IdExp`, `IntExp` 和 `OpExp`。对于 `BinaryOp`，先处理左右 `Exp`，然后若左右均为 `IntExp`，则进行对应运算后返回一个 `IntExp`，否则将处理后的新建一个 `BinaryOp` 并返回。对于 `UnaryOp`，这里不需要处理，只要把内部的 `Exp` 处理后返回即可。对于 `IdExp`, `IntExp` 和 `OpExp`，直接返回一个复制即可。

在 `Program *constantPropagate(Program *root)` 接口中，先将 `AST` 调用 `minusIntRewrite` 重写，然后再将重写后的 `Program` 用 `ConstantPropagationVisitor` 访问器访问得到处理后的 `Program`。

2.程序解释与运行

要求实现另一个访问者 `ExecutorVisitor` 来运行程序并打印返回值。在此过程中，在访问者中使用一个成员表 `map<string, int> env` 来记住并追踪变量的当前值。以及使用 `value` 来记录表达式计算得到的值，用 `returnValue` 记录 `return` 表达式的返回值。

与其他Visitor类似的地方不再赘述。

首先是 `Assign` 节点，先对 `Exp` 求值，然后若 `left` 为 `IdExp`（在SLP里，`left`只会是`IdExp`），则修改成员表 `env` 中记录的值。

对于 `Return` 节点，对 `Exp` 求值后把返回的 `value` 存在 `returnValue` 中返回。

对于 `BinaryOp`，分别对左右求值后计算结果并存到 `value`。

对于 `UnaryOp`，对 `Exp` 求值后再计算结果。

对于 `IdExp`，在 `env` 中查找对应值，找不到则输出 `undefined variable`。变量值存在 `value` 中返回。

对于 `OpExp`，`value=0`。对于 `IntExp`，返回取值，

在 `int execute(Program *root)` 接口中，用 `ExecutorVisitor` 访问器访问后，返回 `returnValue`。

3.git graph

Branches: Show All

Show Remote Branches

Git Graph

Q

Graph	Description	Date	Author	Commit
	master HW1 finished HW1 finished Commit: 270464230023a627ab1d1f4cda79386a1f05ca8c Parents: 55136e8a8a5a996abf68285005f62bcdda1e7421 Author: Starw1sh <23307110269@m.fudan.edu.cn> Committer: Starw1sh <23307110269@m.fudan.edu.cn> Date: Tue Mar 10 2026 22:25:55 GMT+0800 (中国标准时间) HW1 finished	10 Mar 2026 22:32	Starw1sh	0ae3e1b7
		10 Mar 2026 22:25	Starw1sh	27046423
	tinycl2 cmake_install.cmake cmake_install.cmake .ninja_deps .ninja_log build.ninja cmake_install.cmake CMakeCache.txt lib / ast constantPropagation.cc (+147 -2) executor.cc (+128 -2)			
	origin/HEAD origin/master Slight modification HW1 published OK Remove README.md Initialization Initial commit	26 Feb 2026 15:36 23 Feb 2026 00:20 22 Feb 2026 21:32 22 Feb 2026 21:31 22 Feb 2026 21:30 22 Feb 2026 21:28	王晓阳 王晓阳 王晓阳 王晓阳 王晓阳 王晓阳	55136e8a 30b4d729 71e07708 27abdb32 392043f5 a8569e5c

4.测试样例

test1

```
<?xml version="1.0" encoding="UTF-8"?>
<Program>
  <MainMethod>
    <StmList>
      <Assign>
        <IdExp id="x"/>
        <IntExp val="-7"/>
      </Assign>
      <Return>
        <IdExp id="x"/>
      </Return>
    </StmList>
  </MainMethod>
</Program>
```

test2

```
<?xml version="1.0" encoding="UTF-8"?>
<Program>
  <MainMethod>
    <StmList>
      <Assign>
        <IdExp id="x"/>
        <IntExp val="1"/>
      </Assign>
      <Assign>
        <IdExp id="y"/>
        <IntExp val="2"/>
      </Assign>
      <Assign>
        <IdExp id="z"/>
        <BinaryOp>
          <IdExp id="x"/>

```

```

        <OpExp op="+"/>
        <IdExp id="y"/>
    </BinaryOp>
</Assign>
<Return>
    <BinaryOp>
        <IdExp id="x"/>
        <OpExp op="*"/>
        <IdExp id="y"/>
    </BinaryOp>
</Return>
</StmList>
</MainMethod>
</Program>

```

test3

```

<?xml version="1.0" encoding="UTF-8"?>
<Program>
    <MainMethod>
        <StmList>
            <Assign>
                <IdExp id="x"/>
                <IntExp val="1"/>
            </Assign>
            <Assign>
                <IdExp id="y"/>
                <IntExp val="2"/>
            </Assign>
            <Assign>
                <IdExp id="z"/>
                <BinaryOp>
                    <BinaryOp>
                        <IdExp id="x"/>
                        <OpExp op="+"/>
                        <IdExp id="y"/>
                    </BinaryOp>
                    <OpExp op="*"/>
                    <IntExp val="2"/>
                </BinaryOp>
            </Assign>
            <Assign>
                <IdExp id="z"/>
                <BinaryOp>
                    <IdExp id="x"/>
                    <OpExp op="*"/>
                    <IdExp id="y"/>
                </BinaryOp>
            </Assign>
            <Assign>
                <IdExp id="result"/>
                <IdExp id="z"/>
            </Assign>
            <Return>
                <IdExp id="result"/>
            </Return>
        </StmList>
    </MainMethod>
</Program>

```

```
        </Return>
    </StmList>
</MainMethod>
</Program>
```

test4

```
<?xml version="1.0" encoding="UTF-8"?>
<Program>
    <MainMethod>
        <StmList>
            <Assign>
                <IdExp id="x"/>
                <IntExp val="-2"/>
            </Assign>
            <Return>
                <IdExp id="x"/>
            </Return>
        </StmList>
    </MainMethod>
</Program>
```